Demolab — contents

From a capacitor to the LIF neuron

30 May 2026

A neuron's membrane is a thin lipid bilayer separating two salty solutions. Charge piles up on either side, ions leak through embedded channels, and currents are injected by synapses or an experimenter's electrode. Electrically, this is just an RC circuit. The leaky integrate-and-fire (LIF) neuron is what you get when you write Kirchhoff's current law for that circuit and bolt a threshold on top.

The membrane is a capacitor

Two conductors separated by an insulator form a capacitor. The bilayer is the insulator; the intracellular and extracellular fluids are the conductors. The defining relation for a capacitor is

$$Q = C_m V,$$

where $V = V_{\text{in}} - V_{\text{out}}$ is the membrane potential, Q is the charge stored on the inner face, and C_m is the membrane capacitance (typically $\approx 1\mu\text{F/cm}^2$).

Differentiating in time gives the capacitive current, the current required to change the voltage at rate $\frac{\mathrm{d}V}{\mathrm{d}t}$:

$$I_C = \frac{\mathrm{d}Q}{\mathrm{d}t} = C_m \frac{\mathrm{d}V}{\mathrm{d}t}.$$

The membrane is also a leaky resistor

Ion channels punctuate the bilayer. Even at rest some are open, so the membrane has a finite conductance g_L (equivalently, a leak resistance $R_m = 1/g_L$). The leak current is driven by how far the voltage sits from its reversal potential V_{rest}:

$$I_L = g_L(V - V_{\text{rest}}) = \frac{V - V_{\text{rest}}}{R_m}.$$

When $V > V_{\text{rest}}$, the leak current is positive: charge flows outward and pulls V back down. The capacitor and the leak resistor sit in parallel between the inside and the outside of the cell.

Kirchhoff's current law

Now inject an external current I. Charge is conserved at the membrane node: whatever flows in must either charge the capacitor or escape through the leak.

$$I = I_C + I_L = C_m \frac{\mathrm{d}V}{\mathrm{d}t} + \frac{V - V_{\text{rest}}}{R_m}.$$

Multiply through by R_m and define the membrane time constant $\tau_m = R_m C_m$:

$$\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} = -(V - V_{\text{rest}}) + R_m I.$$

This is the subthreshold LIF equation. Nothing has been assumed beyond Kirchhoff's law and linear, passive channels. The dynamics are first-order: any voltage perturbation decays exponentially toward $V_{\text{rest}} + R_m I$ with time constant τ_m.

Adding a spike

Real neurons don't just relax. When V crosses a threshold, voltage-gated sodium channels open, the membrane briefly depolarizes to nearly +40 mV, and potassium channels repolarize it. The

full mechanism is the Hodgkin–Huxley model. LIF throws all of that away and replaces it with a rule:

$$V(t) \geq V_{\text{th}} \quad \Longrightarrow \quad \text{spike at } t, \quad V \leftarrow V_{\text{reset}}.$$

Two parameters, no extra differential equations. Often a refractory period is added during which V is clamped at V_{reset}.

What was thrown away

The derivation makes the model's assumptions explicit:

- **Linear leak.** Real channels are voltage- and time-dependent; g_L is treated as constant.
- **Point neuron.** The whole membrane is collapsed to one node: no dendrites, no axonal delay, no spatial structure.
- **Phenomenological spike.** The action potential is replaced by a reset, so the model says nothing about spike shape, sodium dynamics, or bursting.

In exchange you get a single linear ODE with a reset, which integrates in a handful of lines and scales to networks of thousands of neurons. That is the LIF neuron simulated in exp000: same equation, same RC circuit, now with current injected and a threshold to cross.

The Streamlit spiking-neuron playground

16 June 2026

▸ **Open the live playground:** localhost:8501. Start it with `uv run streamlit run experiments/playground.py` (details below).

Most of this lab runs *batch-style*: a tool command takes a fixed set of arguments, runs a simulation, and drops a directory of artifacts that an experiment then publishes. That is the right shape for a result you want to pin down and cite.

The Streamlit app is the opposite end of that spectrum: a live, interactive surface for *building intuition* before you commit to a run. It re-simulates a single integrate-and-fire neuron on every slider drag and redraws the voltage trace immediately.

What it does

A radio button at the top of the sidebar picks the model, and the app drives the matching `neuron` subcommand: the **same** CLI the experiments run:

- **LIF**, the leaky integrate-and-fire neuron (`neuron lif`), a hard threshold with a spike-and-reset rule. This is the model behind exp000.
- **EIF**, the exponential integrate-and-fire neuron (`neuron eif`), where an explicit exponential term replaces the hard threshold. This is the model behind exp001.

Every parameter is a slider in the sidebar. Most are shared between the two models:

- **Input**, tonic current I and membrane resistance R_m.
- **Membrane**, time constant τ_m, and the resting / reset potentials.
- **Simulation**, duration and the integration timestep Δt.

The spiking mechanism, by contrast, is model-specific: LIF exposes a single hard threshold V_{th}, while EIF exposes the soft threshold V_T, the slope factor Δ_T, and the peak cutoff V_{peak}. As you drag, the app shows the active model, the spike count, and the firing rate in Hz, flagged *spiking* or *silent*.

The LIF membrane equation it integrates is

$$\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} = -(V - V_{\text{rest}}) + R_m I,$$

with a spike-and-reset rule whenever V crosses V_{th}; EIF adds the $\Delta_T \exp((V - V_T)/\Delta_T)$ term and resets when V reaches V_{peak}.

Starting the server

Launch it through `uv` (never call `python` / `streamlit` directly):

```
uv run streamlit run experiments/playground.py
```

Streamlit serves on `http://localhost:8501` by default and opens a browser tab. To pick another port:

```
uv run streamlit run experiments/playground.py --server.port 3001
```

Stop it with Ctrl-C in the terminal.

How it fits the rest of the lab

The playground deliberately sits **outside** the tool → experiment contract: it writes no `config.json` / `output.json` / `manifest.json` of its own to the record, produces no committed

artifacts, and isn't bundled by any experiment runner. It's a thinking tool, not a reproducible run, but it never reimplements the science: each slider settle shells out to the tool CLI and reads the trace back from `temp/neuron/<model>/`, the same files the experiments consume. When a slider configuration produces something worth keeping, reproduce it as a pinned run with the tool instead:

```
uv run python tools/neuron/tool.py eif --current 2.5 --duration 100 --delta-t 2
```

That writes the artifacts an experiment can publish.

The Hodgkin–Huxley membrane, in equations

3 July 2026

Almost all equations, but not quite: a few lines of prose between the blocks so the eye has somewhere to rest. The subject is the four-dimensional system for a space-clamped patch of excitable membrane, the model Hodgkin and Huxley fit to the squid giant axon in 1952 [**1**], with state $\boldsymbol{x} = (V, m, h, n)^\top$: one voltage and three gating variables. Everything below is that one system, taken apart one relation at a time.

Current balance

The membrane is a capacitor in parallel with three voltage-gated conductances [**1, 10**]. Charge conservation across it gives the equation that ties the whole model together: the rate of change of voltage is set by the sum of ionic currents plus whatever is injected:

$$C_m \dot{V} = -\underbrace{\bar{g}_{\mathrm{Na}} m^3 h (V - E_{\mathrm{Na}})}_{I_{\mathrm{Na}}} - \underbrace{\bar{g}_{\mathrm{K}} n^4 (V - E_{\mathrm{K}})}_{I_{\mathrm{K}}} - \underbrace{g_{\mathrm{L}} (V - E_{\mathrm{L}})}_{I_{\mathrm{L}}} + I_{\mathrm{ext}}(t)$$

C_m, membrane capacitance; $\bar{g}_\bullet$, peak conductances; $E_\bullet$, reversal potentials; $m, h, n \in [0, 1]$, gating variables; I_{ext}, injected current.

Each conductance is the peak value scaled by gates raised to a power: the exponents (m^3, n^4) are the number of independent subunits that must all be open at once. The gates are what make the patch excitable; the next few blocks say how they move.

Gating kinetics

Each gate relaxes toward a voltage-dependent target, written two equivalent ways, as forward/backward rates, or as a target value x_∞ approached with time constant τ_x:

$$\dot{x} = \alpha_x(V)(1 - x) - \beta_x(V) x = \frac{x_\infty(V) - x}{\tau_x}(V), \quad x_\infty = \frac{\alpha_x}{\alpha_x + \beta_x}, \quad \tau_x = \frac{1}{\alpha_x + \beta_x}, \quad x \in \{m, h, n\}.$$

$\alpha_x(V)$, $\beta_x(V)$, voltage-dependent opening/closing rate constants; x_∞, steady-state (in)activation; τ_x, its relaxation time constant; $x \in \{m, h, n\}$, the three gating variables.

Rate functions (V in mV, rates in ms^{-1})

The rates themselves are the empirical heart of the model: six functions fit to voltage- clamp data, no first-principles derivation behind them [**1**]. They are what turns the abstract kinetics above into concrete dynamics:

$$\alpha_m(V) = \frac{0.1(V + 40)}{1 - \exp(-(V + 40)/10)}, \quad \beta_m(V) = 4 \exp(-(V + 65)/18),$$

$$\alpha_h(V) = 0.07 \exp(-(V + 65)/20), \quad \beta_h(V) = \frac{1}{1 + \exp(-(V + 35)/10)},$$

$$\alpha_n(V) = \frac{0.01(V + 55)}{1 - \exp(-(V + 55)/10)}, \quad \beta_n(V) = 0.125 \exp(-(V + 65)/80).$$

The α_m and α_n expressions look singular: numerator and denominator both vanish at one voltage, so a naive evaluation returns $0/0$. The function is smooth there anyway: the singularity is removable, and the value is just the limit [**9**]:

$$\alpha_m(V) = \begin{cases} \frac{0.1(V+40)}{1-\exp(-(V+40)/10)} & \text{if } V \neq -40 \\ 1 & \text{if } V = -40 \end{cases} \quad \text{since} \quad \lim_{V \to -40} \alpha_m(V) = 1.$$

Steady-state gating as a Boltzmann sigmoid

Plotted against voltage, each x_∞ is an S-curve, and it is often more convenient to fit it directly as a Boltzmann sigmoid with a half-activation voltage $V_{1/2}$ and a slope factor k than to go through the rate functions [7, 8]. Its derivative has the tidy logistic form that makes the slope at $V_{1/2}$ exactly $1/(4k)$:

$$x_\infty(V) = \frac{1}{1 + \exp\!\big(-\big(V - V_{1/2}\big)/k\big)}, \qquad \frac{\mathrm{d}x_\infty}{\mathrm{d}V} = \frac{x_\infty(1 - x_\infty)}{k}.$$

$V_{1/2}$, half-activation voltage; k, slope factor.

Equilibrium potentials

The reversal potentials $E_\bullet$ that appear in the current balance are not free parameters: they are set by the ionic concentrations either side of the membrane. Nernst gives the single-ion equilibrium; Goldman–Hodgkin–Katz generalises to the current carried by a partially permeant ion and to the resting potential of a membrane permeable to several ions at once [2, 3]:

$$E_S = \frac{RT}{z_S F} \ln \frac{[S]_{\mathrm{out}}}{[S]_{\mathrm{in}}}, \qquad I_S = P_S z_S^2 \frac{V F^2}{RT} \cdot \frac{[S]_{\mathrm{in}} - [S]_{\mathrm{out}} e^{-z_S V F/RT}}{1 - e^{-z_S V F/RT}},$$

$$V_m = \frac{RT}{F} \ln \frac{P_{\mathrm{K}}[\mathrm{K}]_{\mathrm{out}} + P_{\mathrm{Na}}[\mathrm{Na}]_{\mathrm{out}} + P_{\mathrm{Cl}}[\mathrm{Cl}]_{\mathrm{in}}}{P_{\mathrm{K}}[\mathrm{K}]_{\mathrm{in}} + P_{\mathrm{Na}}[\mathrm{Na}]_{\mathrm{in}} + P_{\mathrm{Cl}}[\mathrm{Cl}]_{\mathrm{out}}}.$$

E_S, Nernst potential of ion S; V_m, resting potential of the mixed-permeability membrane; I_S, GHK current carried by S; R, gas constant; T, absolute temperature; F, Faraday constant; z_S, valence of S; $P_\bullet$, membrane permeabilities; $[S]_{\mathrm{in}}$, $[S]_{\mathrm{out}}$, intra- and extracellular concentrations.

Linear stability

With the vector field fully specified, the question of whether the rest state is quiet or poised to spike becomes a matter of eigenvalues [4, 5, 6]. Write the system as $\dot{\boldsymbol{x}} = \boldsymbol{F}(\boldsymbol{x})$ and linearise about a fixed point $\boldsymbol{x}^*$ where $\boldsymbol{F}(\boldsymbol{x}^*) = \boldsymbol{0}$. The Jacobian $\boldsymbol{J} = \partial \boldsymbol{F}/\partial \boldsymbol{x}$ is

$$\boldsymbol{J} = \begin{bmatrix} \partial_V \dot{V} & \partial_m \dot{V} & \partial_h \dot{V} & \partial_n \dot{V} \\ \partial_V \dot{m} & -1/\tau_m & 0 & 0 \\ \partial_V \dot{h} & 0 & -1/\tau_h & 0 \\ \partial_V \dot{n} & 0 & 0 & -1/\tau_n \end{bmatrix}, \qquad \partial_V \dot{V} = -\frac{g_{\mathrm{L}} + \bar{g}_{\mathrm{Na}} m^3 h + \bar{g}_{\mathrm{K}} n^4}{C_m},$$

and the rest state is stable iff every eigenvalue has negative real part: $\det(\boldsymbol{J} - \lambda \boldsymbol{I}) = 0 \Longrightarrow \mathrm{Re}(\lambda_i) < 0$ for all i.

$\boldsymbol{F}$, the model's vector field; $\boldsymbol{x}^*$, a fixed point; $\boldsymbol{J}$, the Jacobian there; λ_i, its eigenvalues; $\partial_a \dot{b}$, the partial of $\dot{b}$ with respect to a.

Synaptic drive and subthreshold response

So far I_{ext} has been an anonymous forcing term. In a network it is synaptic: presynaptic spikes open conductances that pull the membrane toward a synaptic reversal potential [7]. A spike train $\{t_k\}$ arriving through a synapse of reversal E_{syn} adds

$$I_{\mathrm{syn}}(t) = g_{\mathrm{syn}}(t)\big(V - E_{\mathrm{syn}}\big), \qquad g_{\mathrm{syn}}(t) = \bar{g} \sum_{k:\, t_k \le t} \frac{t - t_k}{\tau_s}\, e^{1 - (t - t_k)/\tau_s},$$

where each presynaptic spike contributes an alpha-function conductance transient of peak $\bar{g}$ (the synaptic weight) that rises and decays on the synaptic time constant τ_s. Below threshold, where

the membrane is a leaky integrator $\tau \dot{V} = -(V - V_{\text{rest}}) + RI$ with membrane time constant τ and input resistance R, the voltage is the convolution

$$V(t) = V_{\text{rest}} + \frac{R}{\tau} \int_{-\infty}^{t} e^{-(t-s)/\tau} I(s)\, \mathrm{d}s, \qquad \hat{Z}(\omega) = \frac{R}{1 + \mathrm{i}\omega\tau},$$

with $\hat{Z}(\omega)$ the input impedance, a first-order low-pass whose cutoff sits at $\omega_c = 1/\tau$.

g_{syn}, synaptic conductance; E_{syn}, synaptic reversal potential; $\{t_k\}$, presynaptic spike times; $\bar{g}$, peak conductance (synaptic weight); τ_s, synaptic time constant; τ, R, membrane time constant and input resistance; $\hat{Z}(\omega)$, input impedance; ω_c, low-pass cutoff.

References

1. Hodgkin AL, Huxley AF (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *J Physiol* 117:500–544. doi:10.1113/jphysiol.1952.sp004764
2. Goldman DE (1943). Potential, impedance, and rectification in membranes. *J Gen Physiol* 27:37–60. doi:10.1085/jgp.27.1.37
3. Hodgkin AL, Katz B (1949). The effect of sodium ions on the electrical activity of the giant axon of the squid. *J Physiol* 108:37–77. doi:10.1113/jphysiol.1949.sp004310
4. FitzHugh R (1961). Impulses and physiological states in theoretical models of nerve membrane. *Biophys J* 1:445–466. doi:10.1016/S0006-3495(61)86902-6
5. Nagumo J, Arimoto S, Yoshizawa S (1962). An active pulse transmission line simulating nerve axon. *Proc IRE* 50:2061–2070. doi:10.1109/JRPROC.1962.288235
6. Ermentrout GB, Terman DH (2010). *Mathematical Foundations of Neuroscience.* Springer. doi:10.1007/978-0-387-87708-2
7. Gerstner W, Kistler WM, Naud R, Paninski L (2014). *Neuronal Dynamics.* Cambridge University Press. doi:10.1017/CBO9781107447615
8. Izhikevich EM (2007). *Dynamical Systems in Neuroscience.* MIT Press. doi:10.7551/mitpress/2526.001.0001
9. Sterratt D, Graham B, Gillies A, Willshaw D (2011). *Principles of Computational Modelling in Neuroscience.* Cambridge University Press. doi:10.1017/CBO9780511975899
10. Cole KS, Curtis HJ (1939). Electric impedance of the squid giant axon during activity. *J Gen Physiol* 22:649–670. doi:10.1085/jgp.22.5.649

Guides

6 July 2026

Guides are demolab's **reference layer**: the conventions an agent, and you, can lean on at any moment. They live inside the engine (`demolab-engine/guides/`), so they update when the engine does, and they are always in force rather than run on demand. Type a guide's name in capitals (`RULES`, `SLIDES`) and the agent walks you through it.

Each entry below links to its source on GitHub, so this page can never drift from the guide it points at.

- **RULES**: the contract. The toolchain, the framework/content firewall, the tool and experiment schema, and how to add a tool, an experiment, or a writing.
- **HOUSESTYLE**: how a writing reads. Prose, math, figures, and captions, as numbered H-rules.
- **SLIDES**: deck conventions, the reusable layout catalog, and sizing.
- **STRUCTURE**: the annotated file tree.
- **GLOSSARY**: the vocabulary (tool, experiment, deck, collection, provenance).
- **SUPPORT**: reaching a human.

Their on-demand counterpart, **runbooks**, are documented in a companion article in this collection.

Runbooks

6 July 2026

Runbooks are demolab's **procedure layer**: named, repeatable jobs the agent runs step by step, showing each result and confirming before it moves on. Where a guide is always-on reference, a runbook is invoked when you want it. Type its name in capitals (`LINT`, `DOCTOR`, `GETTING-STARTED`) and the agent opens the runbook and drives it; type `HELP` for the full menu.

Each entry links to its source on GitHub, so this list stays in step with the engine.

- **GETTING-STARTED**: set up a fresh lab, interactively.
- **TOUR**: walk through the repository.
- **LINT**: audit the prose and figures against the house style.
- **DOCTOR**: audit the structure against the rules.
- **RED-TEAM**: attack the result's validity.
- **STEELMAN**: make the strongest case for it.
- **NEXT**: suggest what to run next.
- **GROUND-CLAIMS**: back every claim with a run or a citation.
- **MIGRATE-CODE**: wrap an existing codebase.
- **MIGRATE-STACK**: switch language (MATLAB, R, Julia, Octave).
- **FROM-JUPYTER**: convert a notebook.
- **FROM-PAPER**: reproduce a paper.
- **EMBED-DOCS**: use demolab as a docs subfolder in another project.
- **UPDATE**: vendor the latest engine into your lab.

Their always-on counterpart, **guides**, are documented in a companion article in this collection.

LIF voltage traces and a recurrent network raster

13 May 2026

A single leaky integrate-and-fire (LIF) neuron driven by a steady supra-threshold current should fire periodically; wire 200 of them together with sparse excitation and noisy per-neuron bias and the population should sustain ongoing, irregular firing rather than fall silent or lock into synchrony. We ran both (one neuron, then the network) and report the voltage trace, the raster, and the measured firing rates.

Methods

The LIF membrane voltage decays toward a resting potential and is pushed around by injected current; a threshold crossing emits a spike and resets the voltage.

- **Subthreshold dynamics.** Integrate the leaky-membrane ODE

$$\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} = -(V - V_{\text{rest}}) + R_m I,$$

  with V the membrane potential, τ_m the membrane time constant, V_{rest} the resting potential, R_m the input resistance, and I the injected current.
- **Spike-and-reset.** When $V(t) \geq V_{\text{th}}$, record a spike at t and set $V \leftarrow V_{\text{reset}}$, with V_{th} the threshold and V_{reset} the reset potential.
- **Integration.** Forward Euler at timestep Δt,

$$V \leftarrow V + \frac{\Delta t}{\tau_m}[-(V - V_{\text{rest}}) + R_m I].$$

- **Single-neuron run.** Drive one neuron with a constant supra-threshold tonic current.
- **Network run.** Wire 200 neurons with sparse, all-excitatory recurrence. Each neuron i takes total current $I_i(t) = I_i^{\text{bias}} + s_i(t)$, with noisy bias $I_i^{\text{bias}} \sim \mathcal{N}(\mu, \sigma^2)$ and synaptic current s_i that decays between spikes and is kicked by incoming ones,

$$s_i \leftarrow s_i e^{-\Delta t/\tau_{\text{syn}}} + \sum_{j \in \text{ spiked}} W_{ij},$$

  where τ_{syn} is the synaptic time constant and $W_{ij} = wC_{ij}$ with connectivity $C_{ij} \sim \text{Bernoulli}(p)$, weight w, and no self-connections. Each neuron is held at V_{reset} for a refractory period after spiking.

Parameter values are tabulated below.

Results

With a steady tonic input above threshold, the single neuron fires periodically at 90 Hz. Each upward sweep is the membrane charging through its RC time constant; each vertical line is a reset after a spike.

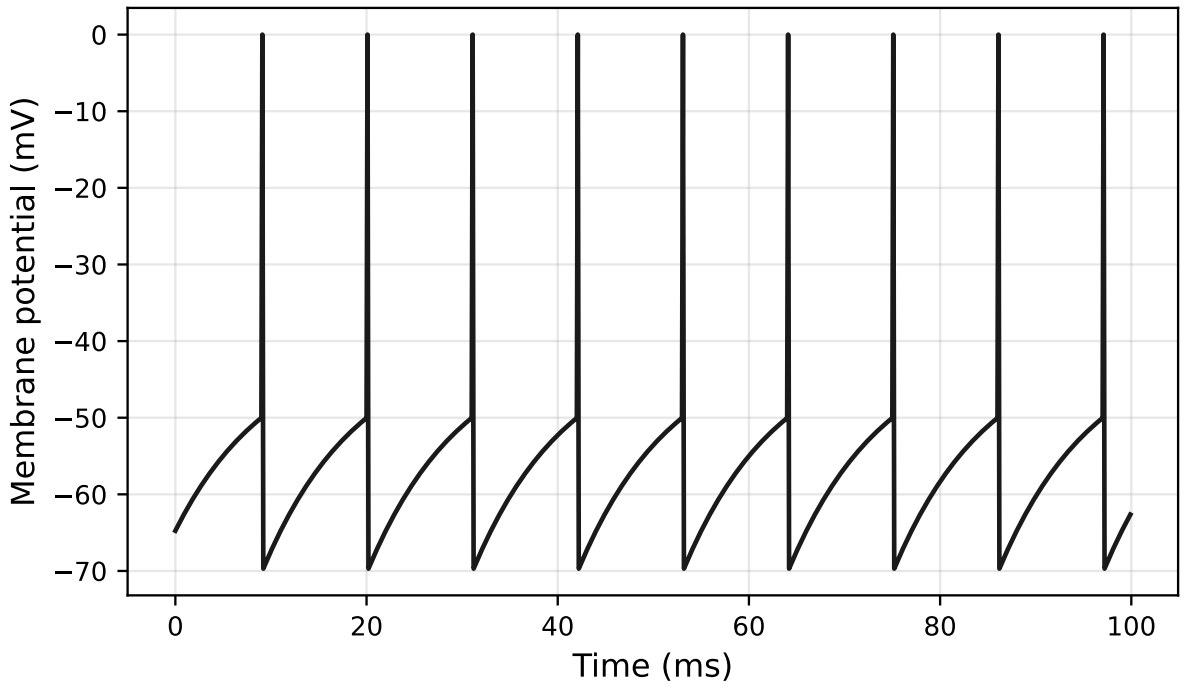

Figure 1: Single LIF neuron under tonic input: membrane potential (mV) against time (ms). The membrane charges toward its steady state and resets at each threshold crossing, giving periodic firing at 90 Hz.

The network sustains ongoing irregular firing: a population mean of 104 Hz, spanning 56–148 Hz across neurons, no silence, no runaway synchrony. The raster shows every spike from every neuron over half a second; the panel beneath tracks the network's mean total input current.

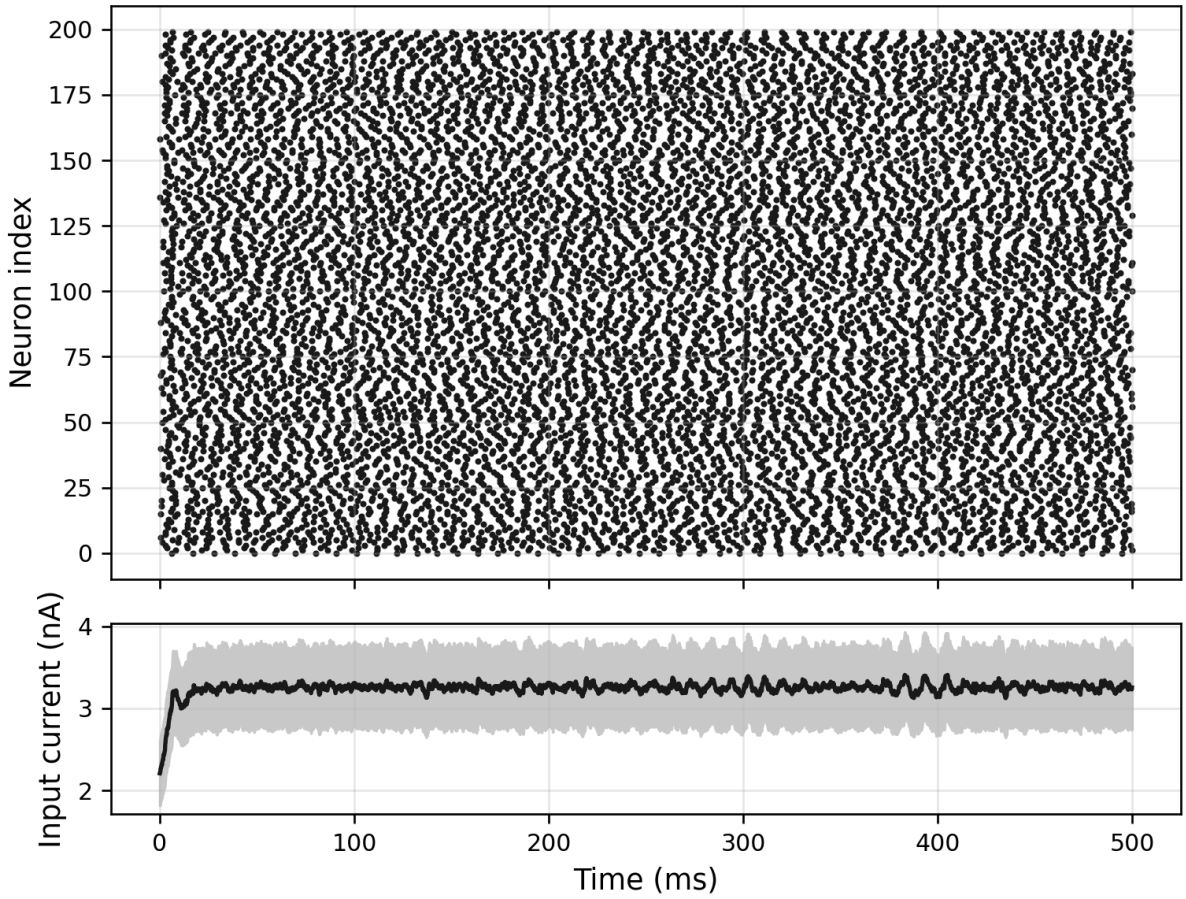

Figure 2: Network raster (top: spike time vs neuron index) and mean input current (bottom, nA vs ms) across 200 recurrently coupled LIF neurons over 500 ms. Mean firing rate 104 Hz; the shaded band is ±1 s.d. of the per-neuron total current across the population.

LIF run parameters

Parameter	Value
current	2.5
duration	100
dt	0.1
tau_m	10
r_m	10
v_rest	−65
v_reset	−70
v_thresh	−50
firing_rate_hz	90

Network run parameters

Parameter	Value
n	200
duration	500
dt	0.1
seed	0
mean_firing_rate_hz	104.2
min_firing_rate_hz	56
max_firing_rate_hz	148

Generated from commit db62207 (uncommitted changes) · 5 July 2026

EIF voltage traces and a recurrent network raster

13 May 2026

The exponential integrate-and-fire (EIF) neuron replaces LIF's hard threshold with an explicit exponential term that models the upswing of a spike. Under the same tonic drive as exp000 it should fire at a different rate than LIF (the exponential accelerates spike initiation, but the upswing itself takes time) while a network of them should behave much like the LIF network with a softened spike onset. We ran the single neuron and the network and compare against exp000.

Methods

The EIF neuron keeps LIF's one-variable spirit but grows an exponential term, so the threshold becomes a soft inflection point rather than a discontinuity.

- **Subthreshold dynamics.** Integrate

$$\tau_m \frac{\mathrm{d}V}{\mathrm{d}t} = -(V - V_{\text{rest}}) + \Delta_T \exp\left(\frac{V - V_T}{\Delta_T}\right) + R_m I,$$

  with V the membrane potential, τ_m the membrane time constant, V_{rest} the resting potential, R_m the input resistance, I the injected current, V_T the soft threshold, and Δ_T the slope factor.
- **Peak-and-reset.** When $V(t) \geq V_{\text{peak}}$, record a spike at t and set $V \leftarrow V_{\text{reset}}$, with V_{peak} the peak cutoff and V_{reset} the reset potential.
- **Threshold behaviour.** V_T is no longer a discontinuity: it is the point at which the exponential term takes over and the voltage blows up on its own. Small Δ_T approximates LIF; larger Δ_T smooths spike initiation.
- **Integration.** Forward Euler at timestep Δt,

$$V \leftarrow V + \frac{\Delta t}{\tau_m}\left[-(V - V_{\text{rest}}) + \Delta_T \exp\left(\frac{V - V_T}{\Delta_T}\right) + R_m I\right].$$

- **Single-neuron run.** Drive one neuron with a constant tonic current.
- **Network run.** Structurally identical to exp000: 200 all-excitatory neurons, sparse p random connectivity, weight w, exponentially-decaying synaptic input with time constant τ_{syn}, noisy per-neuron bias $I_i^{\text{bias}} \sim \mathcal{N}(\mu, \sigma^2)$, and a refractory period, with every neuron now obeying the EIF dynamics above.

Parameter values are tabulated below.

Results

Under the same tonic input, the EIF neuron fires at 60 Hz, against 90 Hz for the LIF neuron in exp000: the exponential term accelerates spike initiation, but the upswing takes time, so the inter-spike period shifts.

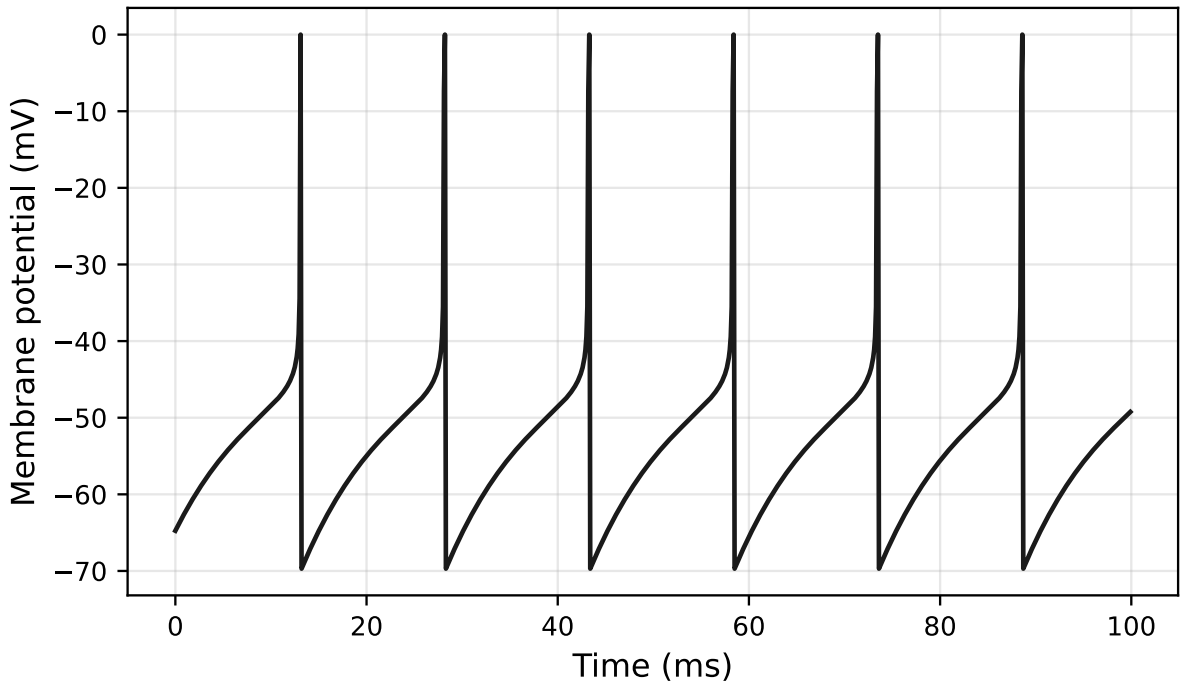

Figure 3: Single EIF neuron under tonic input: membrane potential (mV) against time (ms). The soft exponential threshold produces a curved spike upswing rather than LIF's hard reset, and periodic firing at 60 Hz.

The EIF network sustains irregular firing at a population mean of 69 Hz, spanning 32–98 Hz across neurons, the same asynchronous-irregular regime as the LIF network, shifted in rate.

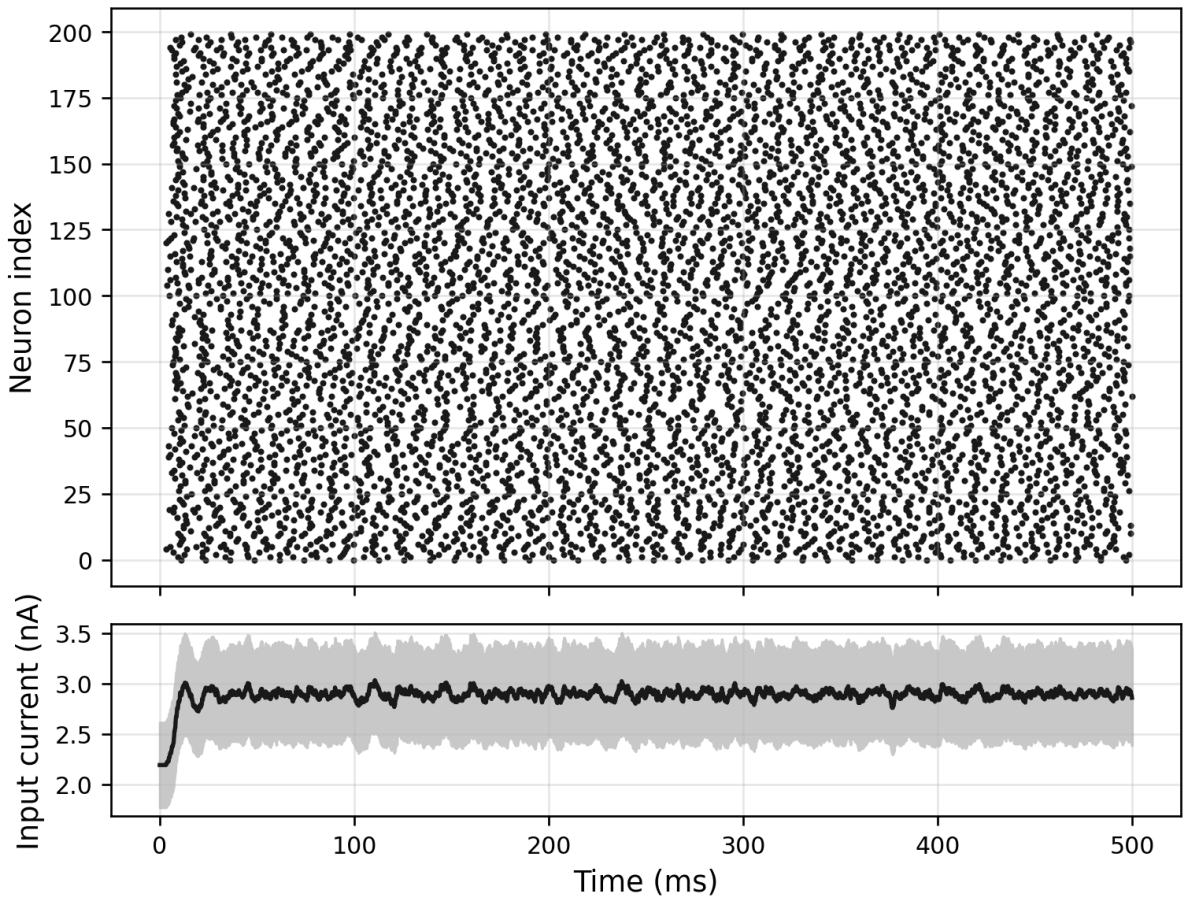

Figure 4: EIF network raster (top: spike time vs neuron index) and mean input current (bottom, nA vs ms) across 200 neurons over 500 ms. Mean firing rate 69 Hz; the shaded band is ±1 s.d. of the per-neuron total current.

EIF run parameters

Parameter	Value
current	2.5
duration	100
dt	0.1
tau_m	10
r_m	10
v_rest	−65
v_reset	−70
v_t	−50
delta_t	2
v_peak	0
firing_rate_hz	60

EIF network run parameters

Parameter	Value
n	200
duration	500
dt	0.1
seed	0
mean_firing_rate_hz	69.11
min_firing_rate_hz	32
max_firing_rate_hz	98

Generated from commit db62207 (uncommitted changes) · 5 July 2026

A passive cartpole in MuJoCo

5 June 2026

The cartpole is the simplest physics-engine model that still looks like something: a cart slides on a frictionless rail with a thin pole hinged on top. With no controller, released from a small offset, the pole should topple under gravity while the cart barely moves, since the rail absorbs almost all of the hinge's reaction force. We ran a four-second passive simulation and recorded the fall.

Methods

The model is a small MJCF cartpole:

- one slide joint for the cart along the x-axis
- one hinge joint for the pole about the y-axis
- damping coefficients 0.05 and 0.005 for the cart and pole
- timestep 0.005 s, simulation length 4 s
- initial pole offset $\theta_0 = 0.15$ rad ($\approx 8.6°$); everything else starts at rest

The simulation is stepped and a frame sampled every 1/60 s into the video below.

Results

[Video — Passive cartpole released from a small offset of $\theta_0 = 0.15$ rad: with no controller the pole topples under gravity while the cart stays almost stationary on the rail. · view the web edition to play.]

The pole crosses 60° from vertical at $t \approx 0.6$ s, ends the run essentially horizontal (final angle 89.5°), and the cart barely moves, only $\approx 3.1 \times 10^{-5}$ m of recoil over the full four seconds, because almost all the angular momentum of the falling pole is absorbed by the rail's reaction force rather than by translating the cart.

Run parameters

Parameter	Value
`theta0`	0.15
`duration`	4
`fps`	60
`fall_time_s`	0.595
`final_angle_deg`	89.52964154260019
`max_cart_displacement_m`	0.00003051021168075732

Generated from commit `60d7d7a` · 3 July 2026

A chaotic double pendulum in MuJoCo

5 June 2026

The double pendulum is the textbook example of a deterministic system that is, in practice, unpredictable: two arms hinged in series have four state variables $(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$ and a Lagrangian with one nonlinear coupling term, and that is enough for trajectories that start arbitrarily close together to separate exponentially. We ran two identical pendulums side by side, started 10^{-3} rad apart, and measured when and how far they diverge.

Methods

The demo runs two double pendulums at once, side by side in the same MuJoCo scene, with identical mass, length, and damping. The only difference is the initial angle of the upper arm,

$$\theta_1^A(0) = 2.0, \quad \theta_1^B(0) = 2.0 + 10^{-3},$$

in radians. The MJCF model is two double-pendulum chains:

- two chains of two hinge arms each, anchored at $x = \pm 0.6$ m
- arms are 0.4 m capsules; a small site marks each tip so its world position can be read
- timestep 0.002 s with the RK4 integrator, important in the chaotic regime, where forward Euler would let small numerical errors swamp the actual divergence
- damping 0, so total energy is conserved (give or take RK4's drift)

Tip separation is measured in each pendulum's *anchor-local* frame (each tip position minus its own anchor) so the constant 1.2 m horizontal offset between the two scenes doesn't contaminate the metric.

Results

For the first two seconds or so the two pendulums trace what looks like the same motion. Around $t \approx 3$ s the tip-to-tip distance crosses the 0.1 m threshold, and from then on they are visibly doing completely different things. The Lyapunov-like blow-up is the whole story.

[Video — Two near-identical double pendulums (started 10^{-3} rad apart) in one MuJoCo scene: they track each other for the first couple of seconds, then diverge visibly once the tip separation passes the 0.1 m threshold. · view the web edition to play.]

The maximum separation hits 1.21 m, basically the full extent of a single pendulum (0.8 m), meaning that at some point one tip is at the top of its swing while the other's is at the bottom. The final separation of 0.56 m is just where they happen to be at $t = 8$ s; with a chaotic system, "where they end up" is meaningless past the divergence time.

Run parameters

Parameter	Value
theta1	2
theta2	0
epsilon	0.001
duration	8
fps	60
separation_threshold	0.1
separation_time_s	2.968
max_separation_m	1.2096309484781214
final_separation_m	0.5648867078342311

Generated from commit 60d7d7a · 3 July 2026